

```

import { useState, useEffect, useCallback } from "react";
import { createClient } from "https://esm.sh/@supabase/supabase-js@2";
import { BarChart, Bar, LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Re

// — SUPABASE —————
const SUPABASE_URL = "https://sienbwtsrenqmkapjjmc.supabase.co";
const SUPABASE_KEY = "sb_publishable_vXJL4hideCViOkp_tyNGyA_XqxbE051";
const sb = createClient(SUPABASE_URL, SUPABASE_KEY);

// — THEME —————
const T = {
  bg:"#0d1117", surface:"#161b22", card:"#1c2128", border:"#30363d",
  accent:"#f97316", green:"#22c55e", red:"#ef4444", blue:"#3b82f6",
  yellow:"#eab308", purple:"#a855f7", cyan:"#06b6d4",
  text:"#e6edf3", muted:"#8b949e",
  b4:"#f97316", b6:"#3b82f6", b7:"#a855f7", b9:"#22c55e",
};
const BRICKS = [
  { id:"4inch", label:'4"', color:T.b4 },
  { id:"6inch", label:'6"', color:T.b6 },
  { id:"7inch", label:'7"', color:T.b7 },
  { id:"9inch", label:'9"', color:T.b9 },
];
const MONTHS = ["Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct","Nov"]
const RM_LIST = ["Flyash","Cement","Sand","Lime"];
const RM_UNITS = { Flyash:"Tonnes", Cement:"Bags", Sand:"Tonnes", Lime:"Bags" };
const PAY_MODES = ["Cash","UPI","Cheque","Bank Transfer"];
const WORKER_ROLES = ["Machine Operator","Helper","Driver","Security","Supervisor"]
const EXP_CATS = ["Electricity","Maintenance","Marketing","Transport","Miscellane

// — UTILS —————
const fmt      = n => "₹" + Number(n).toLocaleString("en-IN");
const fmtN     = n => Number(n).toLocaleString("en-IN");
const today    = () => new Date().toISOString().split("T")[0];
const addDays  = (s,d) => { const dt=new Date(s); dt.setDate(dt.getDate()+d); retu
const weekOf   = s => {
  const d=new Date(s), day=d.getDay(), diff=d.getDate()-day+(day===0?-6:1);
  const mon=new Date(d); mon.setDate(diff);
  const sun=new Date(mon); sun.setDate(mon.getDate()+6);
  return mon.toLocaleDateString("en-IN",{day:"2-digit",month:"short"})
    + " - " +sun.toLocaleDateString("en-IN",{day:"2-digit",month:"short"});
};
const saleDue  = s => s.total_amount - (s.payments||[]).reduce((a,p)=>a+p.amount,0)
const supOwed  = s => (s.bills||[]).reduce((a,b)=>a+b.amount,0) - (s.payments||[])

```

```
// — PRIMITIVES —————

const Card = ({children, style={}, onClick}) => (
  <div onClick={onClick} style={{background:T.card,border:`1px solid ${T.border}`
    borderRadius:14,padding:16,cursor:onClick?"pointer":"default",...style}}>{chi
);

const Hr = () => <div style={{borderTop:`1px solid ${T.border}`,margin:"10px 0"}}>

const Badge = ({children, color=T.accent}) => (
  <span style={{background:color+"22",color,borderRadius:6,padding:"2px 9px",
    fontSize:11,fontWeight:700,border:`1px solid ${color}44`}}>{children}</span>
);

const Stat = ({label,value,sub,color=T.accent,icon}) => (
  <div style={{background:T.card,border:`1px solid ${T.border}`,borderRadius:14,p
    <div style={{display:"flex",justifyContent:"space-between"}}>
      <span style={{fontSize:11,color:T.muted}}>{label}</span>
      <span style={{fontSize:16}}>{icon}</span>
    </div>
    <div style={{fontSize:22,fontWeight:900,color,fontFamily:"Georgia, serif",marg
      {sub}&&<div style={{fontSize:11,color:T.muted,marginTop:2}}>{sub}</div>
    </div>
  </div>
);

const Inp = ({label,value,onChange,type="text",options,placeholder,sm}) => {
  const s={background:T.surface,border:`1px solid ${T.border}`,borderRadius:8,
    padding:sm?"7px 10px":"10px 12px",color:T.text,fontSize:sm?12:14,
    width:"100%",outline:"none",fontFamily:"inherit",boxSizing:"border-box"};
  return (
    <div style={{display:"flex",flexDirection:"column",gap:4}}>
      {label}&&<label style={{fontSize:11,color:T.muted,fontWeight:600}}>{label}</
      {options
        ?<select value={value} onChange={e=>onChange(e.target.value)} style={s}>
          <option value="">Select...</option>
          {options.map(o=><option key={o.value??o} value={o.value??o}>{o.label?
        </select>
        :<input type={type} value={value} onChange={e=>onChange(e.target.value)}
          placeholder={placeholder||""} style={s}/>
      </div>
    );
  };

const Btn = ({children,onClick,color=T.accent,outline=false,sm=false}) => (
  <button onClick={onClick} style={{background:outline?"transparent":color,
    color:outline?color:"#fff",border:`2px solid ${color}`,
    borderRadius:8,padding:sm?"5px 11px":"10px 20px",
    fontSize:sm?11:14,fontWeight:700,cursor:"pointer",
    fontFamily:"inherit",whiteSpace:"nowrap"}}>{children}</button>
);

const IBtn = ({label,onClick,danger=false}) => (
  <button onClick={onClick} style={{background:"none",
```

```

        border:`1px solid ${danger?T.red+"55":T.border}` ,
        borderRadius:6,color:danger?T.red:T.muted,
        fontSize:11,cursor:"pointer",padding:"3px 8px"}}>{label}</button>
    );
const Modal = ({title,children,onClose}) => (
    <div style={{position:"fixed",inset:0,background:"#000c",zIndex:300,
        display:"flex",alignItems:"center",justifyContent:"center",padding:12}}>
        <div style={{background:T.surface,border:`1px solid ${T.border}` ,borderRadius
            padding:22,width:"100%",maxWidth:440,maxHeight:"92vh",overflowY:"auto"}}>
            <div style={{display:"flex",justifyContent:"space-between",alignItems:"cent
                <h3 style={{margin:0,color:T.text,fontSize:16}}>{title}</h3>
                <button onClick={onClose} style={{background:"none",border:"none",color:T
                    </div>
                    {children}
                </div>
            </div>
        </div>
    );
const Tabs = ({tabs,active,onChange}) => (
    <div style={{display:"flex",background:T.surface,borderRadius:10,padding:3,marg
        {tabs.map(t=>(
            <button key={t} onClick={()=>onChange(t)} style={{flex:"0 0 auto",padding:"
                background:active===t?T.accent:"transparent",color:active===t?"#fff":T.mu
                border:"none",borderRadius:8,fontWeight:700,fontSize:11,cursor:"pointer",
                fontFamily:"inherit",whiteSpace:"nowrap"}}>{t}</button>
            )))
        </div>
    );
const SH = ({title,action}) => (
    <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",
        <h2 style={{margin:0,color:T.text,fontSize:19}}>{title}</h2>
        <div style={{display:"flex",gap:8,flexWrap:"wrap"}}>{action}</div>
    </div>
    );
const G2 = ({children,gap=10}) => <div style={{display:"grid",gridTemplateColumns
const Row = ({children,gap=8}) => <div style={{display:"flex",gap,alignItems:"cen
const Loader = () => (
    <div style={{display:"flex",justifyContent:"center",alignItems:"center",padding
        <div style={{color:T.muted,fontSize:14}}><img alt="loading spinner" data-bbox="555 755 570 770" style="vertical-align: middle;"/> Loading...</div>
    </div>
    );
// — DATA HOOK —————
function useData() {
    const [data, setDataRaw] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

```

```

const load = useCallback(async () => {
  setLoading(true);
  try {
    const [inv, rm, prod, workers, salaries, suppliers, customers, sales, expen
      sb.from("inventory").select("*"),
      sb.from("raw_materials").select("*").order("id"),
      sb.from("production").select("*").order("date", {ascending:false}),
      sb.from("workers").select("*").order("id"),
      sb.from("salary_payments").select("*").order("created_at",{ascending:fals
      sb.from("suppliers").select("*").order("id"),
      sb.from("customers").select("*").order("id"),
      sb.from("sales").select("*").order("date",{ascending:false}),
      sb.from("expenses").select("*").order("date",{ascending:false}),
    ]);
    // inventory → object {4inch:N, ...}
    const inventory = {};
    (inv.data||[]).forEach(r=>{ inventory[r.brick_type]=r.qty; });
    setDataRaw({
      inventory,
      rawMaterials: rm.data||[],
      production:   prod.data||[],
      workers:      workers.data||[],
      salaryPayments: (salaries.data||[]).map(s=>({...s, payments: s.payments||
      suppliers:     (suppliers.data||[]).map(s=>({...s, bills:s.bills||[], paym
      customers:     customers.data||[],
      sales:         (sales.data||[]).map(s=>({...s, payments:s.payments||[])}),
      expenses:      expenses.data||[],
    });
    setError(null);
  } catch(e) {
    setError("Failed to load data. Check your connection.");
  }
  setLoading(false);
}, []);

useEffect(()=>{ load(); }, [load]);
return { data, loading, error, reload: load };
}

// — DB HELPERS —————
const DB = {
  // Inventory
  async updateInventory(brickType, qty) {
    await sb.from("inventory").update({qty}).eq("brick_type", brickType);
  },
  async adjustInventory(brickType, delta, currentQty) {
    const newQty = Math.max(0, currentQty + delta);

```

```

    await sb.from("inventory").update({qty: newQty}).eq("brick_type", brickType);
    return newQty;
},

// Raw Materials
async updateRM(id, stock) {
    await sb.from("raw_materials").update({stock}).eq("id", id);
},
async updateRMFull(rm) {
    await sb.from("raw_materials").update({stock:rm.stock, reorder:rm.reorder, co
},

// Production
async addProduction(entry) {
    const {data} = await sb.from("production").insert([
        {
            date: entry.date, brick_type: entry.brickType,
            qty: entry.qty, rejections: entry.rejections,
            materials: entry.materials, added_to_stock: false,
        }
    ]).select().single();
    return data;
},
async updateProduction(id, fields) {
    await sb.from("production").update(fields).eq("id", id);
},
async deleteProduction(id) {
    await sb.from("production").delete().eq("id", id);
},

// Workers
async addWorker(w) {
    const {data} = await sb.from("workers").insert([
        {
            name:w.name, role:w.role, rate
        }
    ]).select().single();
    return data;
},
async updateWorker(w) {
    await sb.from("workers").update({
        name:w.name, role:w.role, rate_per_brick:w.rat
    });
},
async deleteWorker(id) {
    await sb.from("workers").delete().eq("id", id);
},

// Salary
async addSalaryPayment(rec) {
    const {data} = await sb.from("salary_payments").insert([
        {
            week_label: rec.weekLabel, total_bricks: rec.totalBricks, payments: rec.pay
        }
    ]).select().single();
    return data;
},

```

```

async updateSalaryPayment(id, payments) {
  await sb.from("salary_payments").update({payments}).eq("id", id);
},

// Suppliers
async addSupplier(s) {
  const {data} = await sb.from("suppliers").insert([{
    name:s.name, contact:s.contact, material:s.material,
    price_per_unit:Number(s.pricePerUnit), unit:s.unit, terms:s.terms,
    bills:[], payments:[],
  }]).select().single();
  return data;
},
async updateSupplier(id, fields) {
  await sb.from("suppliers").update(fields).eq("id", id);
},
async deleteSupplier(id) {
  await sb.from("suppliers").delete().eq("id", id);
},

// Customers
async addCustomer(c) {
  const {data} = await sb.from("customers").insert([c]).select().single();
  return data;
},
async updateCustomer(c) {
  await sb.from("customers").update({name:c.name,contact:c.contact,location:c.l
},
async deleteCustomer(id) {
  await sb.from("customers").delete().eq("id", id);
},

// Sales
async addSale(s) {
  const {data} = await sb.from("sales").insert([{
    date:s.date, customer:s.customer, brick_type:s.brickType,
    qty:s.qty, price_per_brick:s.pricePerBrick,
    total_amount:s.totalAmount, location:s.location, note:s.note, payments:[],
  }]).select().single();
  return data;
},
async updateSale(id, fields) {
  await sb.from("sales").update(fields).eq("id", id);
},
async deleteSale(id) {
  await sb.from("sales").delete().eq("id", id);
},

```

```
// Expenses
async addExpense(e) {
  const {data} = await sb.from("expenses").insert([
    {
      date:e.date, category:e.category, description:e.description, amount:e.amoun
    }
  ]).select().single();
  return data;
},
async updateExpense(e) {
  await sb.from("expenses").update({date:e.date,category:e.category,description
},
async deleteExpense(id) {
  await sb.from("expenses").delete().eq("id", id);
},
};
```

```
// — SAVING INDICATOR —————
function useSaving() {
  const [saving, setSaving] = useState(false);
  const save = async (fn) => {
    setSaving(true);
    try { await fn(); } finally { setSaving(false); }
  };
  return { saving, save };
}
```

```
// — DASHBOARD —————
function Dashboard({data}) {
  const totalRev = data.sales.reduce((a,s)=>a+s.total_amount,0);
  const totalOut = data.sales.reduce((a,s)=>a+saleDue(s),0);
  const totalStock = Object.values(data.inventory).reduce((a,v)=>a+v,0);
  const lowRM = data.rawMaterials.filter(r=>r.stock<=r.reorder);
  const tod = today();
  const todProd = data.production.filter(p=>p.date===tod).reduce((a,p)=>a+p.qt
  const thisWk = weekOf(tod);
  const wkProd = data.production.filter(p=>weekOf(p.date)===thisWk).reduce((a
  const readyNow = data.production.filter(p=>!p.added_to_stock&&addDays(p.date,
  const monthly = MONTHS.map((m,i)=>({
    month:m,
    revenue: data.sales.filter(s=>new Date(s.date).getMonth()===i).reduce((a,s)=>
    expenses:data.expenses.filter(e=>new Date(e.date).getMonth()===i).reduce((a,e
  )));
  return (
    <div style={{display:"flex",flexDirection:"column",gap:16}}>
      <G2>
        <Stat label="TOTAL REVENUE" value={fmt(totalRev)} color={T.green} ic
        <Stat label="OUTSTANDING" value={fmt(totalOut)} color={T.red} ic
```

```

    <Stat label="TODAY PRODUCTION" value={fmtN(todProd)+" bricks"} color={T.a
    <Stat label="TOTAL STOCK" value={fmtN(totalStock)} color={T.cyan} ic
</G2>
{ (lowRM.length>0||readyNow.length>0)&&(
  <Card style={{borderColor:T.yellow+"55",background:T.yellow+"08"}}>
    <div style={{fontWeight:700,color:T.yellow,marginBottom:8}}>⚠ Alerts</
    {lowRM.map(r=><div key={r.id} style={{fontSize:13,color:T.text,marginBo
      🚫 Low stock: <b>{r.name}</b> - {r.stock} {r.unit} left
    </div>)}
    {readyNow.length>0&&<div style={{fontSize:13,color:T.green}}>
      ✅ {readyNow.length} batch(es) curing done – go to Production to add
    </div>}
  </Card>
)}
<Card>
  <div style={{fontWeight:700,color:T.text,marginBottom:10}}>Monthly Revenu
  <ResponsiveContainer width="100%" height={180}>
    <BarChart data={monthly}>
      <CartesianGrid strokeDasharray="3 3" stroke={T.border}/>
      <XAxis dataKey="month" tick={{fill:T.muted,fontSize:10}}/>
      <YAxis tick={{fill:T.muted,fontSize:9}} tickFormatter={v=>`₹$${v/1000}`}/>
      <Tooltip formatter={v=>fmt(v)} contentStyle={{background:T.surface,bo
      <Bar dataKey="revenue" fill={T.green} radius={[3,3,0,0]} name="Reven
      <Bar dataKey="expenses" fill={T.red} radius={[3,3,0,0]} name="Expen
    </BarChart>
  </ResponsiveContainer>
</Card>
<Card>
  <div style={{fontWeight:700,color:T.text,marginBottom:10}}>Live Stock</di
  {BRICKS.map(b=>(
    <div key={b.id} style={{display:"flex",justifyContent:"space-between",a
      <span style={{color:b.color,fontWeight:600}}>{b.label} Brick</span>
      <span style={{color:T.text,fontWeight:800}}>{fmtN(data.inventory[b.id
    </div>
  )})
</Card>
</div>
);
}

```

```
// — PRODUCTION —
```

```

function Production({data, reload}) {
  const {saving,save} = useSaving();
  const [sub,setSub] = useState("Daily Log");
  const [showAdd,setAdd] = useState(false);
  const [showOut,setOut] = useState(false);
  const [editEntry,setEE]= useState(null);

```



```

const blank = ()=>({date:today(),brickType:"4inch",qty:"1300",rejections:"0",Fl
const [form,setForm]    = useState(blank());
const [soForm,setSO]    = useState({materialId:"",qty:"",reason:"",date:today()})
const fv = v => setForm(p=>({...p,...v}));

```

```

const submitAdd = async () => {
  if(!form.date||!form.brickType||!form.qty) return;
  const mats={Flyash:Number(form.Flyash||0),Cement:Number(form.Cement||0),Sand:
  await save(async()=>{
    await DB.addProduction({date:form.date,brickType:form.brickType,qty:Number(
    // deduct raw materials
    for(const rm of data.rawMaterials) {
      if(mats[rm.name]>0) await DB.updateRM(rm.id, Math.max(0,rm.stock-mats[rm.
    }
    await reload();
  });
  setAdd(false); setForm(blank());
};

```

```

const submitEdit = async () => {
  if(!editEntry) return;
  await save(async()=>{
    await DB.updateProduction(editEntry.id,{
      date:editEntry.date, brick_type:editEntry.brick_type,
      qty:editEntry.qty, rejections:editEntry.rejections,
      materials:editEntry.materials,
    });
    await reload();
  });
  setEE(null);
};

```

```

const delEntry = async id => {
  if(!window.confirm("Delete this entry?")) return;
  await save(async()=>{ await DB.deleteProduction(id); await reload(); });
};

```

```

const addToStock = async p => {
  const net = p.qty - p.rejections;
  await save(async()=>{
    await DB.updateProduction(p.id,{added_to_stock:true});
    await DB.updateInventory(p.brick_type,(data.inventory[p.brick_type]||0)+net
    await reload();
  });
};

```

```

const submitStockOut = async () => {

```

```

    if(!soForm.materialId||!soForm.qty) return;
    const rm = data.rawMaterials.find(r=>r.id===Number(soForm.materialId));
    if(!rm) return;
    await save(async()=>{
        await DB.updateRM(rm.id, Math.max(0,rm.stock-Number(soForm.qty)));
        await reload();
    });
    setOut(false); setSO({materialId:"",qty:"",reason:"",date:today()});
};

const sorted = data.production;
const byWeek = {};
sorted.forEach(p=>{const w=weekOf(p.date);if(!byWeek[w]) byWeek[w]=[];byWeek[w]
const todProd = sorted.filter(p=>p.date===today()).reduce((a,p)=>a+p.qty,0);
const wkLabel = weekOf(today());
const wkProd = sorted.filter(p=>weekOf(p.date)===wkLabel).reduce((a,p)=>a+p.qt
const readyNow= sorted.filter(p=>!p.added_to_stock&&addDays(p.date,5)<=today())
const matTotals=RM_LIST.reduce((acc,m)=>({...acc,[m]:sorted.reduce((a,p)=>a+(p.

return (
    <div style={{display:"flex",flexDirection:"column",gap:14}}>
        <SH title="🏭 Production" action={}>
            {saving&&<span style={{fontSize:11,color:T.muted}}>Saving...</span>}
            <Btn sm outline color={T.red} onClick={()=>setOut(true)}>📦 Stock Out</Btn>
            <Btn sm onClick={()=>{setForm(blank());setAdd(true);}}>+ Add Day</Btn>
        </></>
        <G2>
            <Stat label="TODAY" value={fmtN(todProd)+" bricks"} color={T.accent} icon
            <Stat label="THIS WEEK" value={fmtN(wkProd)+" bricks"} color={T.green} ic
        </G2>

        {readyNow.length>0&&(
            <Card style={{borderColor:T.green+"55",background:T.green+"08"}}>
                <div style={{fontWeight:700,color:T.green,marginBottom:8}}>✅ Curing Dc
                {readyNow.map(p=>{const br=BRICKS.find(b=>b.id===p.brick_type);return(
                    <div key={p.id} style={{display:"flex",justifyContent:"space-between"
                        <span style={{fontSize:13,color:T.text}}>{p.date} · <span style={{c
                            <Btn sm color={T.green} onClick={()=>addToStock(p)}>Add to Stock →<
                        </div>
                    );}})
            </Card>
        )}

        <Tabs tabs={["Daily Log","Weekly View","Materials Used"]} active={sub} onCh

        {sub==="Daily Log"&&sorted.map(p=>{
            const br=BRICKS.find(b=>b.id===p.brick_type);

```

```

const net=p.qty-p.rejections;
const rdy=addDays(p.date,5);
const isRdy=rdy<=today();
const isEdit=editEntry?.id===p.id;
return (
  <Card key={p.id} style={{borderColor:p.added_to_stock?T.green+"33":isRd
    {isEdit?(
      <div style={{display:"flex",flexDirection:"column",gap:10}}>
        <div style={{fontWeight:700,color:T.accent,marginBottom:4}}><img alt="pencil icon" data-bbox="845 200 865 215"/> Edit</div>
        <G2>
          <Inp label="Date" type="date" value={editEntry.date} onChange={
            <Inp label="Brick Type" value={editEntry.brick_type} onChange={
          </G2>
        </G2>
          <Inp label="Produced" type="number" value={editEntry.qty} onCha
            <Inp label="Rejections" type="number" value={editEntry.rejectio
          </G2>
        <div style={{background:T.surface,borderRadius:8,padding:10}}>
          <div style={{fontSize:11,color:T.muted,fontWeight:600,marginBot
            <G2>{RM_LIST.map(m=><Inp key={m} label={m+" (" +RM_UNITS[m]+" )"}
          </div>
          <Row><Btn sm color={T.green} onClick={submitEdit}><img alt="checkmark icon" data-bbox="745 445 765 465"/> Save</Btn><E
        </div>
      ): (
        <>
          <div style={{display:"flex",justifyContent:"space-between",alignI
            <Row gap={8}>
              <span style={{fontWeight:700,color:T.text}}><img alt="calendar icon" data-bbox="725 560 745 575"/> {p.date}</span>
              <Badge color={br?.color}>{br?.label}</Badge>
              {p.added_to_stock?<Badge color={T.green}>In Stock ✓</Badge>
                :isRdy?<Badge color={T.cyan}>Ready</Badge>
                :<Badge color={T.yellow}>Curing till {rdy}</Badge>
            </Row>
            <Row gap={5}>
              <IBtn label="<img alt="pencil icon" data-bbox="435 695 455 715"/>" onClick={()=>setEE({...p,materials:{...p.mat
                <IBtn label="<img alt="trash icon" data-bbox="435 715 455 735"/>" onClick={()=>delEntry(p.id)} danger/>
            </Row>
          </div>
        <div style={{display:"grid",gridTemplateColumns:"1fr 1fr 1fr",gap
          {[["PRODUCED",p.qty,T.text],[["REJECTED",p.rejections,T.red],[["N
            <div key={1} style={{background:T.surface,borderRadius:8,padd
              <div style={{fontSize:9,color:T.muted}}>{1}</div>
              <div style={{fontWeight:900,color:c,fontSize:18}}>{fmtN(v)}
            </div>
          </div>
        </div>
      {p.materials&&Object.values(p.materials).some(v=>v>0)}&&(

```



```

    <div key={r.id} style={{background:T.surface,borderRadius:10,paddin
      <div style={{fontSize:11,color:T.muted}}>{r.name}</div>
      <div style={{fontWeight:800,color:T.accent,fontSize:20}}>{matTota
      <div style={{fontSize:11,color:T.muted}}>Stock left: {r.stock} {r
      <div style={{marginTop:6,background:T.border,borderRadius:99,heig
        <div style={{background:r.stock<=r.reorder?T.red:T.green,height
        </div>
      </div>
    </div>
  )})</G2>
</Card>
{sorted.filter(p=>p.materials&&Object.values(p.materials).some(v=>v>0))
  const br=BRICKS.find(b=>b.id===p.brick_type);
  return (
    <Card key={p.id} style={{padding:"12px 14px"}}>
      <div style={{display:"flex",justifyContent:"space-between",margin
        <span style={{fontWeight:600,color:T.text}}>{p.date}</span>
        <Badge color={br?.color}>{br?.label} · {fmtN(p.qty)} bricks</Ba
      </div>
      <div style={{display:"flex",flexWrap:"wrap",gap:6}}>
        {RM_LIST.map(m=>p.materials[m]>0&&<div key={m} style={{backgrou
        </div>
      </div>
    </Card>
  );
}
})}
</>
)}

```

```

{showAdd&&<Modal title="17 Add Daily Production" onClose={()=>setAdd(false)
  <div style={{display:"flex",flexDirection:"column",gap:12}}>
    <G2>
      <Inp label="Date" type="date" value={form.date} onChange={v=>fv({date
      <Inp label="Brick Type" value={form.brickType} onChange={v=>fv({brick
    </G2>
    <G2>
      <Inp label="Bricks Produced" type="number" value={form.qty} onChange=
      <Inp label="Rejections" type="number" value={form.rejections} onChang
    </G2>
    {form.qty&&<div style={{background:T.green+"18",borderRadius:8,padding:
      <span style={{color:T.muted,fontSize:13}}>Net Good Bricks</span>
      <span style={{color:T.green,fontWeight:800}}>{fmtN(Math.max(0,Number(
    </div>}}
  <div style={{background:T.surface,borderRadius:10,padding:12}}>
    <div style={{fontSize:12,fontWeight:700,color:T.text,marginBottom:10}
      <G2>{data.rawMaterials.map(r=><Inp key={r.id} label={r.name+" ("+r.un
    </div>
    <div style={{fontSize:12,color:T.muted,background:T.surface,borderRadiu
    <Btn onClick={submitAdd} color={T.accent}>{saving?"Saving..":"Save Produ

```

```

    </div>
  </Modal>}

  {showOut&&<Modal title="🧱 Stock Out" onClose={()=>setOut(false)}>
    <div style={{display:"flex",flexDirection:"column",gap:12}}>
      <div style={{fontSize:13,color:T.muted}}>Remove raw material manually (
      <Inp label="Material" value={soForm.materialId} onChange={v=>setSO(f=>(
      <Inp label="Quantity to Remove" type="number" value={soForm.qty} onChan
      <Inp label="Reason (optional)" value={soForm.reason} onChange={v=>setSO
      <Btn onClick={submitStockOut} color={T.red}>{saving?"Saving...":"Remove f
    </div>
  </Modal>}
</div>
);
}

// — SALARIES —————
function Salaries({data,reload}) {
  const {saving,save} = useSaving();
  const [sub,setSub] = useState("This Week");
  const [showAddW,setAddW] = useState(false);
  const [editW,setEditW] = useState(null);
  const [payModal,setPay] = useState(null);
  const [editPayId,setEPI] = useState(null);
  const [wForm,setWF] = useState({name:"",role:"",ratePerBrick:""});

  const calc = (worker,bricks) => Math.round(worker.rate_per_brick*bricks);
  const prodByWeek = {};
  data.production.forEach(p=>{const w=weekOf(p.date);prodByWeek[w]=(prodByWeek[w]
  const thisWk = weekOf(today());
  const thisWkBricks = prodByWeek[thisWk]||0;
  const totalPaid = data.salaryPayments.reduce((a,w)=>a+w.payments.reduce((b,p

  const openPay = (weekLabel,bricks) => {
    const wa={};
    data.workers.forEach(w=>{wa[w.id]=String(calc(w,bricks));});
    setPay({weekLabel,bricks,wa});
  };

  const submitPay = async () => {
    if(!payModal) return;
    const payments=data.workers.map(w=>({workerId:w.id,rate:w.rate_per_brick,earn
    await save(async()=>{ await DB.addSalaryPayment({weekLabel:payModal.weekLabel
    setPay(null);
  };

  const addWorker = async () => {
    if(!wForm.name||!wForm.ratePerBrick) return;
    await save(async()=>{ await DB.addWorker({name:wForm.name,role:wForm.role,rat

```

```

    setAddW(false); setWF({name:"",role:"",ratePerBrick:""});
  };
const saveEditW = async () => {
  await save(async()=>{ await DB.updateWorker({...editW,ratePerBrick:editW.rate
    setEditW(null);
  });
const delWorker = async id => {
  if(!window.confirm("Remove this worker?")) return;
  await save(async()=>{ await DB.deleteWorker(id); await reload(); });
};
const updatePaid = async (weekId,payments) => {
  await save(async()=>{ await DB.updateSalaryPayment(weekId,payments); await re
};

return (
  <div style={{display:"flex",flexDirection:"column",gap:14}}>
    <SH title="👷 Salaries" action={(<
      {saving&&<span style={{fontSize:11,color:T.muted}}>Saving...</span>}
      <Btn sm onClick={()=>setAddW(true)}>+ Worker</Btn>
    </>)}>
    <Stat label="TOTAL SALARY PAID" value={fmt(totalPaid)} color={T.blue} icon=
    <Tabs tabs={["This Week","Workers","History"]} active={sub} onChange={setSu

    {sub==="This Week"&&(
      <>
        <Card style={{borderColor:T.accent+"44"}}>
          <div style={{fontWeight:700,color:T.text,marginBottom:12}}>Week: {thi
          <G2 gap={10}>
            <Stat label="BRICKS THIS WEEK" value={fmtN(thisWkBricks)} color={T.
            <Stat label="TOTAL TO PAY" value={fmt(data.workers.reduce((a,w)=>a+
          </G2>
          <div style={{marginTop:14}}>
            {data.workers.map(w=>(
              <div key={w.id} style={{display:"flex",justifyContent:"space-betw
                <div>
                  <div style={{fontWeight:600,color:T.text}}>{w.name}</div>
                  <div style={{fontSize:12,color:T.muted}}>{w.role} · ₹{w.rate_
                </div>
                <div style={{textAlign:"right"}}>
                  <div style={{fontWeight:800,color:T.green}}>{fmt(calc(w,thisW
                  <div style={{fontSize:11,color:T.muted}}>{fmtN(thisWkBricks)}
                </div>
              </div>
            )))
          </div>
          <div style={{marginTop:14}}><Btn color={T.green} onClick={()=>openPay
        </Card>

```

```

{Object.entries(prodByWeek).filter(([w])=>w!==thisWk&&!data.salaryPayme
  <Card key={week} style={{borderColor:T.yellow+"44"}}>
    <div style={{display:"flex",justifyContent:"space-between",alignIte
      <div>
        <div style={{fontWeight:600,color:T.yellow}}>🕒 Unpaid: {week}<
        <div style={{fontSize:13,color:T.muted}}>{fmtN(bricks)} bricks
      </div>
      <Btn sm color={T.yellow} onClick={()=>openPay(week,bricks)}>Pay N
    </div>
  </Card>
)}
</>
)}

```

```

{sub==="Workers"&&data.workers.map(w=>(
  <Card key={w.id}>
    {editW?.id===w.id?(
      <div style={{display:"flex",flexDirection:"column",gap:10}}>
        <Inp label="Name" value={editW.name} onChange={v=>setEditW(e=>({...
        <Inp label="Role" value={editW.role} onChange={v=>setEditW(e=>({...
        <Inp label="Rate Per Brick (₹)" type="number" value={editW.rate_per
        <Row><Btn sm color={T.green} onClick={saveEditW}>✅ Save</Btn><Btn
      </div>
    ): (
      <div style={{display:"flex",justifyContent:"space-between",alignItems
        <div>
          <div style={{fontWeight:700,color:T.text}}>{w.name}</div>
          <div style={{fontSize:12,color:T.muted}}>{w.role}</div>
          <div style={{fontSize:13,marginTop:4}}>Rate: <span style={{color:
        </div>
        <Row gap={5}><IBtn label="✏️" onClick={()=>setEditW({...w)}}/><IBtn
      </div>
    )}
  </Card>
)})

```

```

{sub==="History"&&data.salaryPayments.map(week=>{
  const totalP=week.payments.reduce((a,p)=>a+p.paid,0);
  const totalE=week.payments.reduce((a,p)=>a+p.earned,0);
  const isEdit=editPayId===week.id;
  return (
    <Card key={week.id}>
      {isEdit?(
        <div style={{display:"flex",flexDirection:"column",gap:10}}>
          <div style={{fontWeight:700,color:T.accent,marginBottom:4}}>✏️ Ec
          {week.payments.map((p,i)=>{
            const w=data.workers.find(x=>x.id===p.workerId);

```



```

        return (
          <div key={i} style={{display:"flex",alignItems:"center",gap:8}}
            <span style={{color:T.text,fontSize:13,minWidth:70}}>{w?.na}</span>
            <span style={{fontSize:11,color:T.muted}}>Earned {fmt(p.earned)}</span>
            <div style={{flex:1}}>
              <Inp type="number" value={p.paid} onChange={v=>{
                const newP=[...week.payments];newP[i]={...p,paid:Number(v)};
                updatePaid(week.id,newP);
              }} sm/>
            </div>
          </div>
        );
      }
    }
    <Btn sm color={T.green} onClick={()=>setEPI(null)}>Done 

```

{showAddWorker?<Modal title="+ Add Worker" onClose={()=>setAddWorker(false)}>
 <div style={{display:"flex",flexDirection:"column",gap:12}}>
 <Inp label="Worker Name" value={wForm.name} onChange={v=>setWF(f=>({...f,name:v})}></Inp>
 <Inp label="Role" value={wForm.role} onChange={v=>setWF(f=>({...f,role:v})}></Inp>
 <Inp label="Rate Per Brick (₹)" type="number" value={wForm.ratePerBrick}></Inp>
 <div style={{background:T.surface,borderRadius:8,padding:10,fontSize:12}}>
 <Btn onClick={addWorker}>{saving?"Saving...":"Add Worker"}</Btn>
 </div>
 </div>
</Modal>

```



```

{payModal?<Modal title="💰 Record Weekly Salary" onClose={()=>setPay(null)}>
 <div style={{display:"flex",flexDirection:"column",gap:12}}>
 <div style={{background:T.surface,borderRadius:8,padding:12}}>

```


```

```

        <div style={{fontSize:12,color:T.muted}}>Week</div>
        <div style={{fontWeight:700,color:T.text}}>{payModal.weekLabel}</div>
        <div style={{fontSize:12,color:T.muted,marginTop:4}}>Bricks: <span st
    </div>
    <div style={{fontSize:12,color:T.muted,fontWeight:600}}>Auto-calculated
    {data.workers.map(w=>(
        <div key={w.id}>
            <div style={{fontSize:11,color:T.muted,marginBottom:4}}>{w.name} ({
                <Inp type="number" value={payModal.wa[w.id]||""} onChange={v=>setPa
            </div>
        )}}
    <div style={{background:T.green+"18",borderRadius:8,padding:12,display:
        <span style={{color:T.muted}}>Total Paying</span>
        <span style={{color:T.green,fontWeight:900,fontSize:18}}>{fmt(Object.
    </div>
    <Btn onClick={submitPay} color={T.green}>{saving?"Saving...": "✅ Confirm
    </div>
    </Modal>}
    </div>
    );
}

```

// — SALES & CUSTOMERS —————

```

function CustomersSales({data,reload}) {
    const {saving,save} = useSaving();
    const [sub,setSub] = useState("Sales");
    const [showSaleM,setSaleM] = useState(false);
    const [showCustM,setCustM] = useState(false);
    const [editSale,setES] = useState(null);
    const [editCust,setEC] = useState(null);
    const [addPayTo,setAPT] = useState(null);
    const [editPay,setEP] = useState(null);
    const [filterMon,setFM] = useState("");
    const [sForm,setSF] = useState({date:today(),customer:"",brickType:"4inch",qty:
    const [cForm,setCF] = useState({name:"",contact:"",location:"",note:""});
    const [payForm,setPF]= useState({date:today(),amount:"",mode:"Cash",note:""});
    const [ePF,setEPF] = useState({date:"",amount:"",mode:"Cash",note:""});
    const sf = v => setSF(p=>({...p,...v}));

    const submitSale = async () => {
        if(!sForm.date||!sForm.customer||!sForm.qty||!sForm.pricePerBrick) return;
        const total=Number(sForm.qty)*Number(sForm.pricePerBrick);
        await save(async()=>{
            await DB.addSale({...sForm,qty:Number(sForm.qty),pricePerBrick:Number(sForm
            await DB.adjustInventory(sForm.brickType,-Number(sForm.qty),data.inventory[
            await reload();
        });
    };
}

```

```

    setSaleM(false);
    setSF({date:today(),customer:"",brickType:"4inch",qty:"",pricePerBrick:"",loc
};
const saveSaleEdit = async () => {
    await save(async()=>{
        await DB.updateSale(editSale.id,{date:editSale.date,customer:editSale.custo
        await reload();
    });
    setES(null);
};
const delSale = async id => {
    if(!window.confirm("Delete sale?")) return;
    await save(async()=>{ await DB.deleteSale(id); await reload(); });
};
const submitPay = async () => {
    if(!payForm.amount||!payForm.date) return;
    const sale=data.sales.find(s=>s.id===addPayTo);
    if(!sale) return;
    const newPays=[...(sale.payments||[]),{id:Date.now(),...payForm,amount:Number
    await save(async()=>{ await DB.updateSale(sale.id,{payments:newPays}); await
    setAPT(null); setPF({date:today(),amount:"",mode:"Cash",note:""});
};
const saveEditPay = async () => {
    const {saleId,payId}=editPay;
    const sale=data.sales.find(s=>s.id===saleId);
    if(!sale) return;
    const newPays=sale.payments.map(p=>p.id===payId?{...p,...ePF,amount:Number(eP
    await save(async()=>{ await DB.updateSale(saleId,{payments:newPays}); await r
    setEP(null);
};
const delPay = async (saleId,payId) => {
    const sale=data.sales.find(s=>s.id===saleId);
    if(!sale) return;
    const newPays=sale.payments.filter(p=>p.id!==payId);
    await save(async()=>{ await DB.updateSale(saleId,{payments:newPays}); await r
};
const addCust = async () => {
    if(!cForm.name) return;
    await save(async()=>{ await DB.addCustomer(cForm); await reload(); });
    setCustM(false); setCF({name:"",contact:"",location:"",note:""});
};
const saveEditCust = async () => {
    await save(async()=>{ await DB.updateCustomer(editCust); await reload(); });
    setEC(null);
};
const delCust = async id => {
    if(!window.confirm("Delete customer?")) return;

```

```

    await save(async()=>{ await DB.deleteCustomer(id); await reload(); });
  };

const filtered=filterMon?data.sales.filter(s=>new Date(s.date).getMonth()===Num
const sorted=[...filtered].sort((a,b)=>b.date.localeCompare(a.date));
const totalRev=data.sales.reduce((a,s)=>a+s.total_amount,0);
const totalOut=data.sales.reduce((a,s)=>a+saleDue(s),0);
const pcol={Cash:T.green,UPI:T.blue,Cheque:T.purple,"Bank Transfer":T.cyan};

return (
  <div style={{display:"flex",flexDirection:"column",gap:14}}>
    <SH title="💰 Sales & Customers" action={<>
      {saving&&<span style={{fontSize:11,color:T.muted}}>Saving...</span>}
      <Btn sm outline onClick={()=>setCustM(true)}>+ Customer</Btn>
      <Btn sm onClick={()=>setSaleM(true)}>+ Sale</Btn>
    </></>
    <G2>
      <Stat label="TOTAL REVENUE" value={fmt(totalRev)} color={T.green} icon="💰"/>
      <Stat label="OUTSTANDING" value={fmt(totalOut)} color={T.red} icon="📄"/>
    </G2>
    <Tabs tabs={["Sales","By Customer","Dues"]} active={sub} onChange={setSub}/>

    {sub==="Sales"&&(
      <>
        <Inp label="Filter by Month" value={filterMon} onChange={setFM} options
        {sorted.map(s=>{
          const br=BRICKS.find(b=>b.id===s.brick_type);
          const paid=(s.payments||[]).reduce((a,p)=>a+p.amount,0);
          const due=s.total_amount-paid;
          const isEdit=editSale?.id===s.id;
          return (
            <Card key={s.id} style={{borderColor:due>0?T.yellow+"44":T.green+"3
              {isEdit?(
                <div style={{display:"flex",flexDirection:"column",gap:10}}>
                  <div style={{fontWeight:700,color:T.accent,marginBottom:4}}>🎨
                  <Inp label="Date" type="date" value={editSale.date} onChange=
                  <Inp label="Customer" value={editSale.customer} onChange={v=>
                  <Inp label="Brick Type" value={editSale.brick_type} onChange=
                </G2>
                  <Inp label="Qty" type="number" value={editSale.qty} onChange
                  <Inp label="Price/Brick (₹)" type="number" value={editSale.
                </G2>
                  <Inp label="Total Amount (₹) - override if needed" type="numb
                  <Inp label="Location" value={editSale.location} onChange={v=>
                  <Inp label="Note" value={editSale.note} onChange={v=>setES(e=
                  <Row><Btn sm color={T.green} onClick={saveSaleEdit}>✅ Save</
                </div>
              )
            )
          )
        })}
      </>
    )
  )

```

```

):(
  <>
  <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",width:"100%"}}>
    <div>
      <div style={{fontWeight:700,color:T.text}}>{s.customer}</div>
      <div style={{fontSize:12,color:T.muted}}><div style={{display:"flex",alignItems:"center",gap:5}}>
        <div style={{marginTop:6,display:"flex",flexWrap:"wrap",gap:5}}>
          <Badge color={br?.color}>{br?.label}</Badge>
          <span style={{fontSize:13,color:T.muted}}>{fmtN(s.qty)}</span>
        </div>
        {s.note}&&<div style={{fontSize:12,color:T.muted,marginTop:5}}>{s.note}</div>
      </div>
      <div style={{textAlign:"right"}}>
        <div style={{fontWeight:900,color:T.green,fontSize:20}}>{due}>0?<div style={{color:T.red,fontWeight:700,fontSize:12}}>{due}</div>:0</div>
      </div>
    </div>
    { (s.payments||[]).length>0&&(
      <div style={{marginBottom:10}}>
        <div style={{fontSize:11,color:T.muted,marginBottom:5}}>P
        {s.payments.map(p=>(
          <div key={p.id} style={{display:"flex",justifyContent:"space-between",width:"100%"}}>
            {editPay?.saleId===s.id&&editPay?.payId===p.id?(
              <div style={{display:"flex",gap:5,flex:1,alignItems:"center"}}>
                <div style={{flex:1,minWidth:90}}><Inp type="date" value={p.date}/>
                <div style={{flex:1,minWidth:80}}><Inp type="number" value={p.amount}/>
                <div style={{flex:1,minWidth:90}}><Inp value={ePF}/>
                <Btn sm color={T.green} onClick={saveEditPay}>✓</Btn>
                <Btn sm outline onClick={()=>setEP(null)}>✕</Btn>
              </div>
            ):(
              <>
                <span style={{color:T.text}}>{p.date} · <Badge color={p.status}>{p.status}</Badge>
                <div style={{display:"flex",gap:5,alignItems:"center"}}>
                  <span style={{color:T.green,fontWeight:700}}>+</span>
                  <IBtn label="✎" onClick={()=>setEP({saleId:s.id,payId:p.id,amount:p.amount,ePF:p.ePF})}></IBtn>
                  <IBtn label="🗑️" onClick={()=>delPay(s.id,p.id)}></IBtn>
                </div>
              </>
            )}
          </div>
        )}
      </div>
    )}
  </div>
)}
</Row>
<Btn sm color={T.blue} onClick={()=>setAPT(s.id);setPF({date:s.date,amount:s.amount,ePF:s.ePF})}>
<IBtn label="✎ Edit Sale" onClick={()=>setES({...s})}/>

```

```

        <IBtn label="🗑 Delete" onClick={()=>delSale(s.id)} danger/
      </Row>
    </>
  )}
</Card>
);
}}
</>
)}

{sub==="By Customer"&&data.customers.map(c=>{
  const cs=data.sales.filter(s=>s.customer===c.name);
  const totalB=cs.reduce((a,s)=>a+s.total_amount,0);
  const totalD=cs.reduce((a,s)=>a+saleDue(s),0);
  return (
    <Card key={c.id} style={{borderColor:totalD>0?T.yellow+"44":T.border}}>
      {editCust?.id===c.id?(
        <div style={{display:"flex",flexDirection:"column",gap:10}}>
          <Inp label="Name" value={editCust.name} onChange={v=>setEC(e=>({.
            <Inp label="Phone" value={editCust.contact} onChange={v=>setEC(e=
            <Inp label="Location" value={editCust.location} onChange={v=>setE
            <Inp label="Note" value={editCust.note} onChange={v=>setEC(e=>({.
            <Row><Btn sm color={T.green} onClick={saveEditCust}>✅ Save</Btn>
          </div>
        ): (
          <>
            <div style={{display:"flex",justifyContent:"space-between",alignI
              <div>
                <div style={{fontWeight:700,color:T.text,fontSize:15}}>{c.nam
                <div style={{fontSize:12,color:T.muted}}>{c.contact?"☎ "+c.c
                {c.note&&<div style={{fontSize:12,color:T.muted,marginTop:2}}
                <div style={{fontSize:13,marginTop:6}}>Total: <span style={{c
              </div>
              <div style={{textAlign:"right"}}>
                {totalD>0?<><div style={{fontWeight:900,color:T.red,fontSize:
              </div>
            </div>
            <div style={{marginTop:10,display:"flex",gap:6}}>
              <IBtn label="✎ Edit" onClick={()=>setEC({...c})}/>
              <IBtn label="🗑 Delete" onClick={()=>delCust(c.id)} danger/>
            </div>
          </>
        )}
      </Card>
    );
  }}
)}

```

```

{sub==="Dues"&&(
  <>
  <div style={{fontSize:13,color:T.muted}}>All pending collections</div>
  {[...data.sales].filter(s=>saleDue(s)>0).sort((a,b)=>saleDue(b)-saleDue
    const br=BRICKS.find(b=>b.id===s.brick_type);
    const paid=(s.payments||[]).reduce((a,p)=>a+p.amount,0);
    const due=saleDue(s);
    return (
      <Card key={s.id} style={{borderColor:T.red+"44"}}>
        <div style={{display:"flex",justifyContent:"space-between",alignI
          <div>
            <div style={{fontWeight:700,color:T.text}}>{s.customer}</div>
            <div style={{fontSize:12,color:T.muted}}>📅 {s.date}{s.locati
            <div style={{fontSize:13,marginTop:4}}><Badge color={br?.colo
            <div style={{fontSize:13,marginTop:6}}>Sale: <span style={{co
          </div>
          <div style={{textAlign:"right"}}><div style={{fontWeight:900,co
        </div>
        {(s.payments||[]).length>0&&<<Hr/>{s.payments.map(p=><div key={p
          <div style={{marginTop:10}}><Btn sm color={T.blue} onClick={()=>{
        </Card>
      );
    })}
    {data.sales.filter(s=>saleDue(s)>0).length===0&&<Card><div style={{text
  </>
})}

```

```

{showSaleM&&<Modal title="📝 New Sale" onClose={()=>setSaleM(false)}>
  <div style={{display:"flex",flexDirection:"column",gap:12}}>
    <Inp label="Date" type="date" value={sForm.date} onChange={v=>sf({date:
    <Inp label="Customer" value={sForm.customer} onChange={v=>sf({customer:
    <Inp label="Brick Type" value={sForm.brickType} onChange={v=>sf({brickT
  <G2>
    <Inp label="Qty (bricks)" type="number" value={sForm.qty} onChange={v
    <Inp label="Price/Brick (₹)" type="number" value={sForm.pricePerBrick
  </G2>
  {sForm.qty&&sForm.pricePerBrick&&<div style={{background:T.green+"18",b
  <Inp label="Location / City" value={sForm.location} onChange={v=>sf({lo
  <Inp label="Note (optional)" value={sForm.note} onChange={v=>sf({note:v
  <div style={{fontSize:12,color:T.muted,background:T.surface,borderRadiu
  <Btn onClick={submitSale} color={T.green}>{saving?"Saving...":}✅ Record
  </div>
</Modal>}

```

```

{showCustM&&<Modal title="+ New Customer" onClose={()=>setCustM(false)}>
  <div style={{display:"flex",flexDirection:"column",gap:12}}>
    <Inp label="Customer Name" value={cForm.name} onChange={v=>setCF(f=>({.

```

```

        <Inp label="Phone Number" value={cForm.contact} onChange={v=>setCF(f=>{
        <Inp label="Location / City" value={cForm.location} onChange={v=>setCF(
        <Inp label="Note" value={cForm.note} onChange={v=>setCF(f=>({...f,note:
        <Btn onClick={addCust}>{saving?"Saving...":"Add Customer"}</Btn>
    </div>
  </Modal>}

  {addPayTo&&<Modal title="👤 Add Payment" onClose={()=>setAPT(null)}>
    <div style={{display:"flex",flexDirection:"column",gap:12}}>
      {(()=>{const s=data.sales.find(x=>x.id===addPayTo);const due=s?saleDue(
        <div style={{background:T.surface,borderRadius:8,padding:10}}>
          <div style={{fontSize:12,color:T.muted}}>Customer: <b style={{color:
          <div style={{fontSize:12,color:T.muted}}>Sale Total: <b style={{col
          <div style={{fontSize:13,color:T.red,fontWeight:700}}>Outstanding:
        </div>
      );}})
    <Inp label="Date" type="date" value={payForm.date} onChange={v=>setPF(f
    <Inp label="Amount Received (₹)" type="number" value={payForm.amount} o
    <Inp label="Payment Mode" value={payForm.mode} onChange={v=>setPF(f=>({
    <Inp label="Note (e.g. 1st installment)" value={payForm.note} onChange=
    <Btn onClick={submitPay} color={T.blue}>{saving?"Saving...":"Save Payment
  </div>
  </Modal>}
</div>
);
}

// — SUPPLIERS —————
function Suppliers({data,reload}) {
  const {saving,save} = useSaving();
  const [sel,setSel] = useState(null);
  const [showAdd,setAdd]= useState(false);
  const [showBill,setBill]=useState(false);
  const [showPay,setPay]=useState(false);
  const [editB,setEB] = useState(null);
  const [editP,setEP] = useState(null);
  const [editS,setES] = useState(null);
  const [sF,setSF]=useState({name:"",contact:"",material:"",pricePerUnit:"",unit:
  const [bF,setBF]=useState({date:today(),qty:"",amount:"",note:""});
  const [pF,setPF]=useState({date:today(),amount:"",mode:"Cash",note:""});

  const addS=async()=>{if(!sF.name) return;await save(async()=>{await DB.addSuppl
  const saveES=async()=>{await save(async()=>{await DB.updateSupplier(editS.id,{n
  const delS=async id=>{if(!window.confirm("Delete supplier?")) return;await save

  const withSup=(id,fn)=>{
    const sup=data.suppliers.find(s=>s.id===id);

```



```

    if(!sup) return null;
    return fn(sup);
};

const addBill=async()=>{
    if(!bF.date||!bF.amount) return;
    withSup(sel,async sup=>{
        const bills=[...(sup.bills||[]),{id:Date.now(),...bF,qty:Number(bF.qty||0),
        await save(async()=>{await DB.updateSupplier(sel,{bills});await reload();})
    });
    setBill(false);setBF({date:today(),qty:"",amount:"",note:""});
};

const saveEB=async()=>{
    withSup(sel,async sup=>{
        const bills=sup.bills.map(b=>b.id===editB.id?editB:b);
        await save(async()=>{await DB.updateSupplier(sel,{bills});await reload();})
    });
    setEB(null);
};

const delBill=async id=>{
    withSup(sel,async sup=>{
        const bills=sup.bills.filter(b=>b.id!==id);
        await save(async()=>{await DB.updateSupplier(sel,{bills});await reload();})
    });
};

const addPay=async()=>{
    if(!pF.date||!pF.amount) return;
    withSup(sel,async sup=>{
        const payments=[...(sup.payments||[]),{id:Date.now(),...pF,amount:Number(pF
        await save(async()=>{await DB.updateSupplier(sel,{payments});await reload()
    });
    setPay(false);setPF({date:today(),amount:"",mode:"Cash",note:""});
};

const saveEP=async()=>{
    withSup(sel,async sup=>{
        const payments=sup.payments.map(p=>p.id===editP.id?editP:p);
        await save(async()=>{await DB.updateSupplier(sel,{payments});await reload()
    });
    setEP(null);
};

const delPay=async id=>{
    withSup(sel,async sup=>{
        const payments=sup.payments.filter(p=>p.id!==id);
        await save(async()=>{await DB.updateSupplier(sel,{payments});await reload()
    });
};

```

```

if(sel!==null){
  const sup=data.suppliers.find(s=>s.id===sel);
  if(!sup){setSel(null);return null;}
  const totalB=(sup.bills||[]).reduce((a,b)=>a+b.amount,0);
  const totalP=(sup.payments||[]).reduce((a,p)=>a+p.amount,0);
  const balance=totalB-totalP;
  return (
    <div style={{display:"flex",flexDirection:"column",gap:14}}>
      <div style={{display:"flex",gap:10,alignItems:"center"}}>
        <button onClick={()=>setSel(null)} style={{background:"none",border:"no
        {editS?.id===sup.id?(
          <div style={{flex:1,display:"flex",flexDirection:"column",gap:8}}>
            <Inp value={editS.name} onChange={v=>setES(e=>({...e,name:v}))} pla
            <Inp value={editS.contact} onChange={v=>setES(e=>({...e,contact:v}))}
            <Inp label="Price/Unit (₹)" type="number" value={editS.price_per_un
            <Row><Btn sm color={T.green} onClick={saveES}>✅ Save</Btn><Btn sm
          </div>
        ):
        <div style={{flex:1,display:"flex",justifyContent:"space-between",ali
          <div><h2 style={{margin:0,color:T.text,fontSize:18}}>{sup.name}</h2>
          <Row gap={5}><IBtn label="✎" onClick={()=>setES({...sup})}/><IBtn
        </div>
        </div>
      </div>
    <Card>
      <div style={{display:"grid",gridTemplateColumns:"1fr 1fr",gap:10,margin
        {[["Material",sup.material],["Rate","₹"+sup.price_per_unit+"/"+sup.un
          <div key={1}><div style={{fontSize:10,color:T.muted}}>{1}</div><div>
        </div>
      </div>
      <div style={{display:"grid",gridTemplateColumns:"1fr 1fr 1fr",gap:8}}>
        {[["BILLED",totalB,T.yellow],["PAID",totalP,T.green],["BALANCE",balan
          <div key={1} style={{textAlign:"center"}}><div style={{fontSize:10,
        </div>
      </div>
    </Card>

    <div style={{display:"flex",justifyContent:"space-between",alignItems:"ce
      <div style={{fontWeight:700,color:T.text}}>📄 Bills ({(sup.bills||[]).1
      <Btn sm color={T.yellow} onClick={()=>setBill(true)}>+ Add Bill</Btn>
    </div>
    [...(sup.bills||[])].sort((a,b)=>b.date.localeCompare(a.date)).map(b=>(
      <Card key={b.id} style={{padding:"12px 14px"}}>
        {editB?.id===b.id?(
          <div style={{display:"flex",flexDirection:"column",gap:8}}>
            <Inp label="Date" type="date" value={editB.date} onChange={v=>set

```

```

        <Inp label="Qty" type="number" value={editB.qty} onChange={v=>set
        <Inp label="Amount (₹)" type="number" value={editB.amount} onChan
        <Inp label="Note" value={editB.note} onChange={v=>setEB(e=>({...e
        <Row><Btn sm color={T.green} onClick={saveEB}>✅ Save</Btn><Btn s
    </div>
  ): (
    <div style={{display:"flex",justifyContent:"space-between",alignIte
      <div><div style={{fontWeight:600,color:T.text}}>📅 {b.date}</div>
      <Row gap={5}><div style={{fontWeight:800,color:T.yellow,fontSize:
    </div>
  )}
</Card>
))}

<div style={{display:"flex",justifyContent:"space-between",alignItems:"ce
  <div style={{fontWeight:700,color:T.text}}>📄 Payments ({(sup.payments|
  <Btn sm color={T.green} onClick={()=>setPay(true)}>+ Record Payment</Bt
</div>
{[... (sup.payments||[])].sort((a,b)=>b.date.localeCompare(a.date))}.map(p=
  <Card key={p.id} style={{padding:"12px 14px"}}>
    {editP?.id===p.id?(
      <div style={{display:"flex",flexDirection:"column",gap:8}}>
        <Inp label="Date" type="date" value={editP.date} onChange={v=>set
        <Inp label="Amount (₹)" type="number" value={editP.amount} onChan
        <Inp label="Mode" value={editP.mode} onChange={v=>setEP(e=>({...e
        <Inp label="Note" value={editP.note} onChange={v=>setEP(e=>({...e
        <Row><Btn sm color={T.green} onClick={saveEP}>✅ Save</Btn><Btn s
      </div>
    ): (
      <div style={{display:"flex",justifyContent:"space-between",alignIte
        <div><div style={{fontWeight:600,color:T.text}}>📅 {p.date} · <Ba
        <Row gap={5}><div style={{fontWeight:800,color:T.green,fontSize:1
      </div>
    )}
  </Card>
))}

{showBill&&<Modal title="📄 Add Bill" onClose={()=>setBill(false)}><div s
{showPay&&<Modal title="📄 Record Payment" onClose={()=>setPay(false)}><d
</div>
);
}

return (
  <div style={{display:"flex",flexDirection:"column",gap:14}}>
    <SH title="👤 Suppliers" action={<Btn sm onClick={()=>setAdd(true)}>+ Add S
    <div style={{fontSize:13,color:T.muted}}>Tap a supplier to view bills & pay

```

```

    {data.suppliers.map(s=>{const bal=supOwed(s);return(
      <Card key={s.id} onClick={()=>setSel(s.id)}>
        <div style={{display:"flex",justifyContent:"space-between",alignItems:"
          <div>
            <div style={{fontWeight:700,color:T.text,fontSize:15}}>{s.name}</di
            <div style={{fontSize:12,color:T.muted}}><img alt="phone icon" data-bbox="645 140 665 155"/> {s.contact} · {s.materi
            <div style={{fontSize:13,marginTop:5}}>Rate: <span style={{color:T.
          </div>
          <div style={{textAlign:"right"}}>
            {bal>0?<><div style={{fontWeight:900,color:T.red,fontSize:18}}>{fmt
            <div style={{color:T.muted,fontSize:11,marginTop:4}}>Tap to open →<
          </div>
        </div>
      </Card>
    )}})}
    {showAdd&&<Modal title="+ New Supplier" onClose={()=>setAdd(false)}><div st
  </div>
);
}

```

// — RAW MATERIALS —————

```

function RawMaterials({data,reload}) {
  const {saving,save} = useSaving();
  const [showIn,setIn]=useState(false);
  const [editRM,setERM]=useState(null);
  const [form,setForm]=useState({materialId:"",qty:"",note:""});
  const submitIn=async()=>{
    if(!form.materialId||!form.qty) return;
    const rm=data.rawMaterials.find(r=>r.id===Number(form.materialId));
    if(!rm) return;
    await save(async()=>{await DB.updateRM(rm.id,rm.stock+Number(form.qty));await
      setIn(false);setForm({materialId:"",qty:"",note:""});
    });
  const saveEdit=async()=>{
    await save(async()=>{await DB.updateRMFull(editRM);await reload();});
    setERM(null);
  };
  return (
    <div style={{display:"flex",flexDirection:"column",gap:14}}>
      <SH title="📦 Raw Materials" action={<Btn sm onClick={()=>setIn(true)}>+ St
      {data.rawMaterials.map(r=>{const low=r.stock<=r.reorder,pct=Math.min(100,(r
        <Card key={r.id} style={{borderColor:low?T.red+"55":T.border}}>
          {editRM?.id===r.id?(
            <div style={{display:"flex",flexDirection:"column",gap:10}}>
              <div style={{fontWeight:700,color:T.text}}>Edit: {r.name}</div>
              <Inp label="Current Stock" type="number" value={editRM.stock} onCha
              <Inp label="Reorder Level" type="number" value={editRM.reorder} onC

```

```

        <Inp label="Cost Per Unit (₹)" type="number" value={editRM.cost_per}
        <Row><Btn sm color={T.green} onClick={saveEdit}>✅ Save</Btn><Btn s
    </div>
  ): (
    <>
      <div style={{display:"flex",justifyContent:"space-between",alignIte
        <div><div style={{fontWeight:700,color:T.text,fontSize:16}}>{r.na
        <Row gap={8}>
          <div style={{textAlign:"right"}}><div style={{fontSize:28,fontW
            <IBtn label="✎" onClick={()=>setERM({...r})}/>
          </Row>
        </div>
      <div style={{marginTop:10,background:T.border,borderRadius:99,heigh
        {low}&&<div style={{marginTop:8,color:T.red,fontSize:13,fontWeight:6
      </>
    })}
  </Card>
  );)}}
  {showIn&&<Modal title="+ Stock In" onClose={()=>setIn(false)}><div style={{
    </div>
  );
}

```

```

// — INVENTORY —————
function Inventory({data,reload}) {
  const {saving,save} = useSaving();
  const [modal,setM]=useState(false);
  const [form,setForm]=useState({brickType:"",adjustment:"",note:""});
  const submit=async()=>{
    if(!form.brickType||!form.adjustment) return;
    const cur=data.inventory[form.brickType]||0;
    const newQ=Math.max(0,cur+Number(form.adjustment));
    await save(async()=>{await DB.updateInventory(form.brickType,newQ);await relo
    setM(false);setForm({brickType:"",adjustment:"",note:""});
  };
  return (
    <div style={{display:"flex",flexDirection:"column",gap:14}}>
      <SH title="🧱 Inventory" action=<Btn sm outline onClick={()=>setM(true)}>A
      <Stat label="TOTAL STOCK" value={fmtN(Object.values(data.inventory).reduce(
        {BRICKS.map(b=>{
          <Card key={b.id}>
            <div style={{display:"flex",justifyContent:"space-between",alignItems:"
              <div><div style={{fontSize:22,fontWeight:900,color:b.color}}>{b.label
              <div style={{textAlign:"right"}}><div style={{fontSize:32,fontWeight:
            </div>
            <div style={{marginTop:10,background:T.border,borderRadius:99,height:8}
          </Card>
        }
      )
    )
  )
}

```

```

    )))
    {modal}&&<Modal title="Adjust Stock" onClose={()=>setM(false)}><div style={{
</div>
    });
}

// — EXPENSES —————
function Expenses({data,reload}) {
  const {saving,save} = useSaving();
  const [modal,setM]=useState(false);
  const [editE,setEE]=useState(null);
  const [form,setForm]=useState({date:today(),category:"",description:"",amount:""});
  const cc={Electricity:T.yellow,Maintenance:T.red,Marketing:T.accent,Transport:T.accent};
  const submit=async()=>{if(!form.date||!form.category||!form.amount) return;await
  const saveEdit=async()=>{await save(async()=>{await DB.updateExpense(editE);await
  const del=async id=>{if(!window.confirm("Delete expense?")) return;await save(async
  const total=data.expenses.reduce((a,e)=>a+e.amount,0);
  const catT=EXP_CATS.map(c=>({name:c,val:data.expenses.filter(e=>e.category===c)
  return (
    <div style={{display:"flex",flexDirection:"column",gap:14}}>
      <SH title="📊 Expenses" action={<Btn sm color={T.red} onClick={()=>setM(true)}>
      <Stat label="TOTAL EXPENSES" value={fmt(total)} color={T.red} icon="📊"/>
      <div style={{display:"flex",flexWrap:"wrap",gap:8}}>{catT.map(c=><div key={c}
      {data.expenses.map(e=>(
        <Card key={e.id} style={{padding:"12px 14px"}}>
          {editE?.id===e.id?(
            <div style={{display:"flex",flexDirection:"column",gap:10}}>
              <Inp label="Date" type="date" value={editE.date} onChange={v=>setEE({date:v})}>
              <Inp label="Category" value={editE.category} onChange={v=>setEE({category:v})}>
              <Inp label="Description" value={editE.description} onChange={v=>setEE({description:v})}>
              <Inp label="Amount (₹)" type="number" value={editE.amount} onChange={v=>setEE({amount:v})}>
              <Row><Btn sm color={T.green} onClick={saveEdit}>✅ Save</Btn><Btn sm color={T.red}
            </div>
          ):()>
            <div style={{display:"flex",justifyContent:"space-between",alignItems:"center",gap:10}}>
              <div><div style={{fontWeight:600,color:T.text}}>{e.description}|<div style={{fontWeight:800,color:T.red}}>{e.category}</div></div>
              <Row gap={6}>
                <div style={{textAlign:"right"}}><div style={{fontWeight:800,color:T.red}}>
                  <IBtn label="✏️" onClick={()=>setEE({...e})}>
                  <IBtn label="🗑️" onClick={()=>del(e.id)} danger/>
                </div>
              </Row>
            </div>
          )}
        </Card>
      )}
    </div>
  )}
  {modal}&&<Modal title="Add Expense" onClose={()=>setM(false)}><div style={{display:"flex",flexDirection:"column",gap:14}}>
    <div style={{display:"flex",flexDirection:"column",gap:14}}>
      <Inp label="Date" type="date" value={form.date} onChange={v=>setForm({date:v})}>
      <Inp label="Category" value={form.category} onChange={v=>setForm({category:v})}>
      <Inp label="Description" value={form.description} onChange={v=>setForm({description:v})}>
      <Inp label="Amount (₹)" type="number" value={form.amount} onChange={v=>setForm({amount:v})}>
      <Row><Btn sm color={T.green} onClick={submit}>✅ Save</Btn><Btn sm color={T.red}
    </div>
  )}
  </div>

```

```
);  
}
```

```
// — REPORTS —
```

```
function Reports({data}) {  
  const monthly=MONTHS.map((m,i)=>{  
    const revenue =data.sales.filter(s=>new Date(s.date).getMonth()===i).reduce(  
    const expenses =data.expenses.filter(e=>new Date(e.date).getMonth()===i).redu  
    const salaries =data.salaryPayments.reduce((a,w)=>{  
      try{if(new Date(w.week_label.split("-")[0].trim()+` ${new Date().getFullYear  
    },0);  
    const produced=data.production.filter(p=>new Date(p.date).getMonth()===i).red  
    return {month:m,revenue,revenue,expenses,salaries,produced,profit:revenue-expenses-sa  
  });  
  const best=[...monthly].sort((a,b)=>b.revenue-a.revenue)[0];  
  const totalR=data.sales.reduce((a,s)=>a+s.total_amount,0);  
  const totalC=data.expenses.reduce((a,e)=>a+e.amount,0)+data.salaryPayments.redu  
  const locs=[...new Set(data.sales.map(s=>s.location))].map(loc=>({loc,revenue:d  
  return (  
    <div style={{display:"flex",flexDirection:"column",gap:16}}>  
      <h2 style={{margin:0,color:T.text}}> Reports & Analytics</h2>  
      <G2>  
        <Stat label="NET PROFIT" value={fmt(totalR-totalC)} color={T.green} icon=  
        <Stat label="BEST MONTH" value={best?.month||"-"} sub={fmt(best?.revenue|  
      </G2>  
      <Card>  
        <div style={{fontWeight:700,color:T.text,marginBottom:12}}>Revenue vs Pro  
        <ResponsiveContainer width="100%" height={190}>  
          <LineChart data={monthly.filter(m=>m.revenue>0)}>  
            <CartesianGrid strokeDasharray="3 3" stroke={T.border}/>  
            <XAxis dataKey="month" tick={{fill:T.muted,fontSize:10}}/>  
            <YAxis tick={{fill:T.muted,fontSize:9}} tickFormatter={v=>`₹$${v/1000}}  
            <Tooltip formatter={v=>fmt(v)} contentStyle={{background:T.surface,bo  
            <Line type="monotone" dataKey="revenue" stroke={T.green} strokeWidth=  
            <Line type="monotone" dataKey="profit" stroke={T.accent} strokeWidth=  
          </LineChart>  
        </ResponsiveContainer>  
      </Card>  
      <Card>  
        <div style={{fontWeight:700,color:T.text,marginBottom:12}}>Monthly Bricks  
        <ResponsiveContainer width="100%" height={160}>  
          <BarChart data={monthly.filter(m=>m.produced>0)}>  
            <CartesianGrid strokeDasharray="3 3" stroke={T.border}/>  
            <XAxis dataKey="month" tick={{fill:T.muted,fontSize:10}}/>  
            <YAxis tick={{fill:T.muted,fontSize:9}}/>  
            <Tooltip contentStyle={{background:T.surface,border:`1px solid ${T.bo  
            <Bar dataKey="produced" fill={T.cyan} radius={[3,3,0,0]} name="Bricks
```

```

        </BarChart>
      </ResponsiveContainer>
    </Card>
    <Card>
      <div style={{fontWeight:700,color:T.text,marginBottom:12}}>📌 Sales by Loc
      {locs.map((l,i)=>(
        <div key={l.loc} style={{display:"flex",justifyContent:"space-between",
          <div><div style={{fontWeight:600,color:T.text}}>{["🥇","🥈","🥉"][i]}
          <div style={{fontWeight:800,color:T.accent,fontSize:18}}>{fmt(l.reven
        </div>
      ))}
    </Card>
    <Card>
      <div style={{fontWeight:700,color:T.text,marginBottom:12}}>Month-by-Month
      {[...monthly].filter(m=>m.revenue>0).sort((a,b)=>b.revenue-a.revenue).map
        <div key={m.month} style={{marginBottom:14}}>
          <div style={{display:"flex",justifyContent:"space-between",marginBott
            <span style={{fontWeight:700,color:T.text}}>{m.month}</span>
            <span style={{color:m.profit>0?T.green:T.red,fontWeight:700}}>Prof
          </div>
          <div style={{fontSize:12,color:T.muted,display:"flex",gap:12,flexWrap
            <span>Revenue: <b style={{color:T.green}}>{fmt(m.revenue)}</b></spa
            <span>Bills: <b style={{color:T.red}}>{fmt(m.expenses)}</b></span>
            <span>Salary: <b style={{color:T.blue}}>{fmt(m.salaries)}</b></span>
            <span>Bricks: <b style={{color:T.cyan}}>{fmtN(m.produced)}</b></spa
          </div>
        </div>
      ))}
    </Card>
  </div>
);
}

```

// — APP SHELL —

```

const NAV=[
  {id:"dashboard",  label:"Home",      icon:"🏠"},
  {id:"inventory",  label:"Stock",      icon:"📦"},
  {id:"production", label:"Production", icon:"🏭"},
  {id:"raw",         label:"Materials", icon:"🪨"},
  {id:"sales",       label:"Sales",     icon:"💰"},
  {id:"salaries",    label:"Salaries",  icon:"👷"},
  {id:"expenses",    label:"Expenses",  icon:"📊"},
  {id:"suppliers",   label:"Suppliers", icon:"🚚"},
  {id:"reports",     label:"Reports",   icon:"📈"},
];

```

```

export default function App() {

```



```

const [tab,setTab]      = useState("dashboard");
const {data,loading,error,reload} =  useData();

if(loading) return (
  <div style={{minHeight:"100vh",background:T.bg,display:"flex",flexDirection:"
    <div style={{width:48,height:48,background:T.accent,borderRadius:14,display
      <div style={{color:T.muted,fontSize:14}}>Loading BrickMaster Pro...</div>
    </div>
  );

if(error) return (
  <div style={{minHeight:"100vh",background:T.bg,display:"flex",flexDirection:"
    <div style={{fontSize:32}}>⚠️</div>
    <div style={{color:T.red,fontWeight:700,fontSize:16}}>Connection Error</div>
    <div style={{color:T.muted,fontSize:13,textAlign:"center"}}>{error}</div>
    <Btn onClick={reload} color={T.blue}>Retry</Btn>
  </div>
);

const p={data,reload};
const render=()=>{
  switch(tab){
    case "dashboard":  return <Dashboard      data={data}/>;
    case "inventory":  return <Inventory      {...p}/>;
    case "production": return <Production     {...p}/>;
    case "raw":        return <RawMaterials   {...p}/>;
    case "sales":      return <CustomersSales {...p}/>;
    case "salaries":   return <Salaries       {...p}/>;
    case "expenses":   return <Expenses       {...p}/>;
    case "suppliers":  return <Suppliers      {...p}/>;
    case "reports":    return <Reports        data={data}/>;
    default: return null;
  }
};

return (
  <div style={{minHeight:"100vh",background:T.bg,fontFamily:"'Segoe UI',sans-se
    <div style={{background:T.surface,borderBottom:`1px solid ${T.border}`,padd
      <div style={{display:"flex",alignItems:"center",gap:12}}>
        <div style={{width:34,height:34,background:T.accent,borderRadius:10,dis
          <div>
            <div style={{fontWeight:800,fontSize:15,color:T.text,fontFamily:"Geor
              <div style={{fontSize:10,color:T.muted}}>Flyash Bricks Business Manag
            </div>
          </div>
        <button onClick={reload} style={{background:"none",border:`1px solid ${T.
      </div>

```

```

<div style={{padding:"18px 14px 100px",maxWidth:700,margin:"0 auto"}}>
  {render()}
</div>
<div style={{position:"fixed",bottom:0,left:0,right:0,background:T.surface,
  {NAV.map(t=>(
    <button key={t.id} onClick={()=>setTab(t.id)} style={{flex:"0 0 auto",d
      <span style={{fontSize:15}}>{t.icon}</span>
      <span style={{fontSize:9,fontWeight:tab===t.id?700:400}}>{t.label}</s
    </button>
  )}}
</div>
</div>
);
}

```